

VisionGuard AI

Implementation & Architecture Document

This document provides the complete technical architecture specification for VisionGuard AI. It covers system architecture, module design, database schema, AI pipeline, security, caching, WebSocket design, deployment, and scaling strategy.

Version	v2.0
Document Type	Implementation & Architecture Document
Platform	VisionGuard AI
Status	Final

IMPLEMENTATION.md

VisionGuard AI

Enterprise Factory Safety Monitoring & Compliance Platform

Table of Contents

1. [Architecture Overview](#1-architecture-overview)
2. [High Level System Architecture](#2-high-level-system-architecture)
3. [Physical Architecture](#3-physical-architecture)
4. [Modular Monolith — Module Breakdown](#4-modular-monolith--module-breakdown)
5. [Clean Architecture — Per Module Structure](#5-clean-architecture--per-module-structure)
6. [Authentication & Authorization](#6-authentication--authorization)
7. [Event Driven Architecture](#7-event-driven-architecture)
8. [AI Worker Architecture](#8-ai-worker-architecture)
9. [Rules Engine](#9-rules-engine)
10. [WebSocket & Real-Time Architecture](#10-websocket--real-time-architecture)
11. [API Design & Versioning](#11-api-design--versioning)
12. [Caching Strategy](#12-caching-strategy)
13. [Database Architecture](#13-database-architecture)
14. [Snapshot Storage Architecture](#14-snapshot-storage-architecture)
15. [Data Retention Policy](#15-data-retention-policy)
16. [Dashboard Architecture](#16-dashboard-architecture)
17. [Analytics Architecture](#17-analytics-architecture)
18. [Notification Architecture](#18-notification-architecture)
19. [Zone Configuration Flow](#19-zone-configuration-flow)
20. [API Rate Limiting](#20-api-rate-limiting)
21. [Security Architecture](#21-security-architecture)
22. [Monitoring & Observability](#22-monitoring--observability)
23. [Audit Trail](#23-audit-trail)
24. [Logging Strategy](#24-logging-strategy)
25. [Testing Strategy](#25-testing-strategy)
26. [CI/CD Pipeline](#26-cicd-pipeline)
27. [Deployment Architecture](#27-deployment-architecture)
28. [Scaling Strategy](#28-scaling-strategy)
29. [Future Integrations](#29-future-integrations)
30. [Technology Stack](#30-technology-stack)
31. [Success Criteria](#31-success-criteria)

1. Architecture Overview

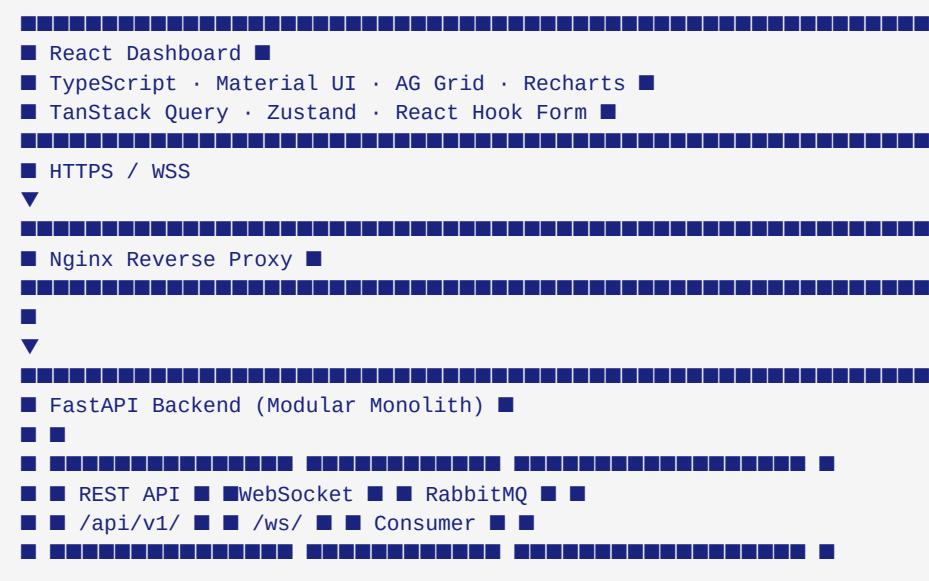
Style

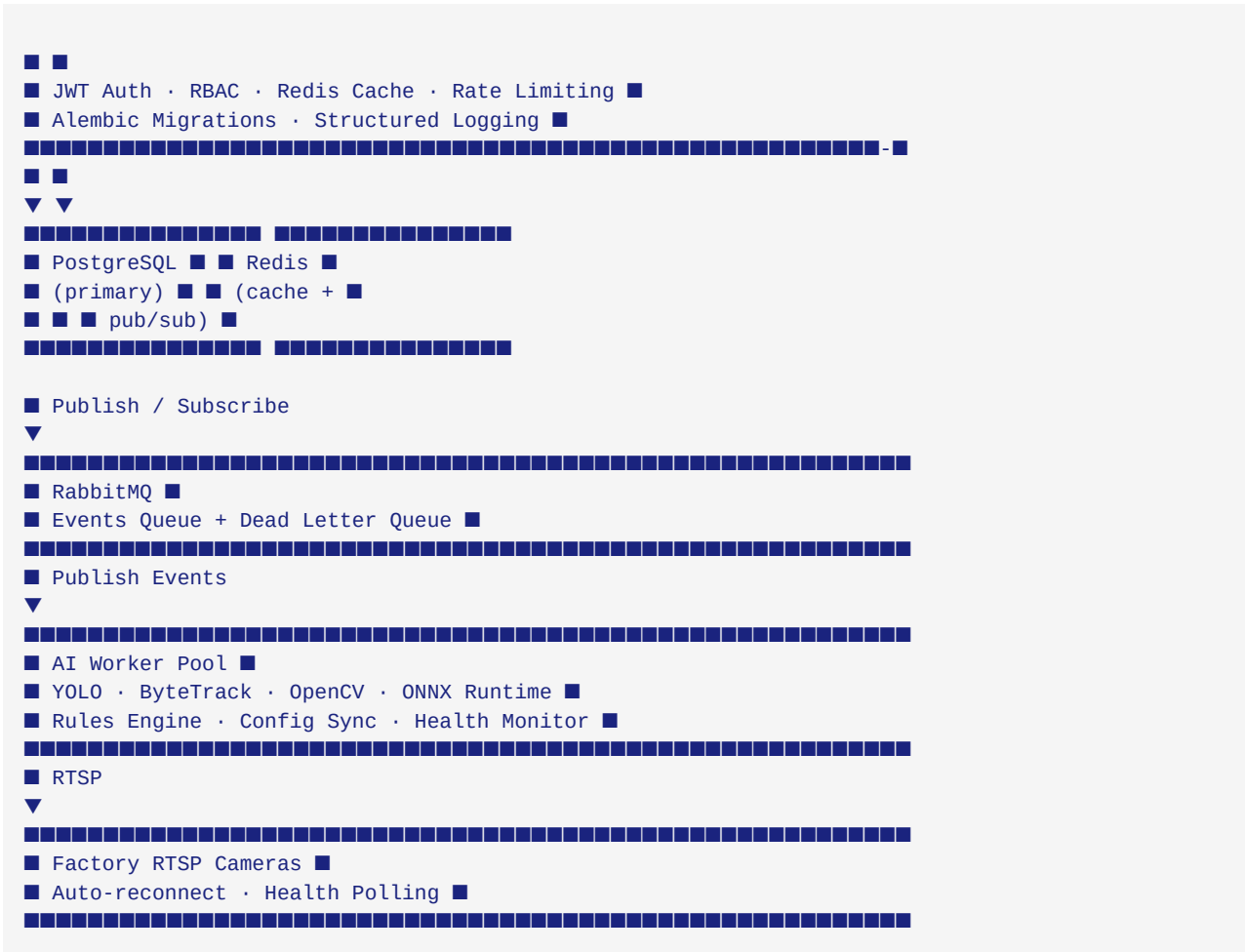
Principle	Choice	Reason
Structure	Modular Monolith	Simple deployment, easy debugging, lower ops cost
Design	Clean Architecture + DDD	Business logic independent of framework/infra
Communication	Event Driven (RabbitMQ)	Decoupled AI layer, zero DB access from workers
Query	CQRS (selective)	Heavy read paths separated from write paths
Future	Microservice-ready	Each module can be extracted independently

Core Principle

> AI Workers **never touch the database**. They only publish events. The backend owns all persistence.

2. High Level System Architecture





3. Physical Architecture

Frontend

Technology: React 18 · TypeScript · Vite · Material UI · AG Grid · Recharts · TanStack Query · Zustand · React Hook Form · Zod

Responsibilities:

- Executive, Safety, Camera, and Analytics dashboards
- Real-time alert feed (WebSocket)
- Zone configuration management
- Report generation (PDF / Excel download)
- User and role management (Admin only)

Backend

Technology: FastAPI · SQLAlchemy 2.0 · Alembic · Pydantic v2 · PostgreSQL · Redis

Responsibilities:

- JWT authentication + refresh token rotation
- Role-based access control (RBAC)
- REST API with versioning, pagination, filtering, sorting
- WebSocket hub for real-time dashboard and alert push
- RabbitMQ consumer — processes AI worker events
- Redis cache management
- Alert lifecycle management
- Notification dispatch
- Analytics aggregation
- PDF + Excel report generation
- Audit logging

AI Worker

Technology: Python · YOLO · OpenCV · ByteTrack · ONNX Runtime · PyTorch · CUDA / CPU

Responsibilities:

- RTSP stream capture per camera
- Frame sampling (2 FPS from 25 FPS)
- Person + PPE detection (YOLO)
- Multi-object tracking (ByteTrack)
- Occupancy counting per zone
- Rules engine evaluation
- Violation snapshot capture
- RabbitMQ event publishing
- Zone config sync (hot-reload on change)
- Model hot-swap (no stream restart)
- Worker heartbeat + health metrics

4. Modular Monolith — Module Breakdown

```
backend/src/modules/
```

```
■■■ identity/ Users, roles, permissions, JWT, sessions
■■■ organization/ Multi-tenant support (future)
■■■ factory/ Factory CRUD, plant head assignment
■■■ zone/ Zone management, occupancy rules, supervisor
■■■ camera/ Camera registry, RTSP config, health status
■■■ worker/ AI worker registry, heartbeat, assignment
■■■ occupancy/ Real-time count logs, peak tracking
■■■ ppe/ Violation records, snapshot references
```

```

■■■■ alerts/ Alert creation, lifecycle, history, assignment
■■■■ notifications/ Email, in-app, web push, Slack, Teams, webhook
■■■■ analytics/ Aggregated KPIs, trends, heatmaps, safety score
■■■■ dashboard/ Dashboard summary APIs (cached)
■■■■ reports/ PDF + Excel generation, download
■■■■ config/ Zone-level detection config, version management
■■■■ audit/ Immutable audit log, compliance export
■■■■ health/ /health/ready + /health/live endpoints

```

Module Isolation Rules

- No module imports from another module's `infrastructure/` layer
- Cross-module communication via:
 - Domain events (async, within process)
 - Application service interfaces (sync, within process)
 - RabbitMQ events (async, AI worker ↔ backend)

5. Clean Architecture — Per Module Structure

```

module/
■■■■ api/
■ ■■■ routes.py FastAPI router
■ ■■■ request_models.py Pydantic input schemas
■ ■■■ response_models.py Pydantic output schemas
■
■■■■ application/
■ ■■■ commands/ Write operations (CreateAlert, ResolveAlert)
■ ■■■ queries/ Read operations (GetAlertList, GetAlertDetail)
■ ■■■ services/ Orchestration between domain + infra
■
■■■■ domain/
■ ■■■ entities.py Core business objects
■ ■■■ value_objects.py Immutable domain concepts
■ ■■■ events.py Domain events raised by entities
■ ■■■ repositories.py Repository interfaces (abstract)
■ ■■■ rules.py Business rule validation
■
■■■■ infrastructure/
■ ■■■ models.py SQLAlchemy ORM models
■ ■■■ repositories.py Concrete repository implementations
■ ■■■ external/ Third-party clients (email, push, etc.)
■
■■■■ tests/
■■■■ unit/
■■■■ integration/

```

Example: Alert Module Flow

```

POST /api/v1/alerts/{id}/resolve
↓
AlertRouter → ResolveAlertCommand

```

```
↓
ResolveAlertHandler
↓
AlertRepository.get(id) ← domain
Alert.resolve(user_id, comment) ← domain rule
AlertRepository.save(alert) ← infrastructure
AlertResolvedEvent published ← domain event
↓
NotificationService.notify_supervisor() ← cross-module via interface
AuditService.log(action) ← cross-module via interface
WebSocketManager.broadcast(alert) ← real-time push
CacheService.invalidate("alert:*") ← cache invalidation
```

6. Authentication & Authorization

JWT Strategy

```
POST /api/v1/auth/login
↓
Credentials validated
↓
Access Token (JWT) → expires in 8 hours
Refresh Token (opaque) → expires in 7 days
↓
Access Token → returned in response body (stored in memory by client)
Refresh Token → returned in HttpOnly cookie (never accessible via JS)
```

Refresh Token Rotation

```
Access Token expires
↓
Client sends Refresh Token (via HttpOnly cookie)
↓
Backend validates token hash against identity.refresh_tokens
↓
Old token revoked (status = Revoked)
New Access Token + new Refresh Token issued
↓
Replay attack: old token used → all tokens for user revoked → force re-login
```

Token Blacklisting on Logout

```
POST /api/v1/auth/logout
↓
Refresh Token revoked in DB
Access Token JTI added to Redis blacklist
```

```
TTL = remaining token lifetime
↓
Every authenticated request checks Redis blacklist
Blacklisted token → 401 Unauthorized
```

Role Based Access Control (RBAC)

Roles: Admin · SafetyOfficer · Supervisor · Viewer

Permissions stored as JSONB per role:

```
{
  "alerts": ["read", "acknowledge", "resolve"],
  "cameras": ["read"],
  "config": [],
  "users": [],
  "audit": [],
  "reports": ["generate", "download"]
}
```

Permission check enforced in the API layer on every request.

Unauthorized access → 403 Forbidden + audit log entry.

7. Event Driven Architecture

Event Flow

```
Camera
↓
AI Detection
↓
Rules Engine Evaluation
↓
Event Created (with snapshot_key, confidence, zone_id, camera_id)
↓
RabbitMQ – main exchange
↓ (on failure → Dead Letter Queue)
Backend Consumer
↓
PostgreSQL (violation + alert created)
↓
Redis cache invalidated
↓
WebSocket broadcast to connected supervisors
↓
Notification dispatched (email + in-app + web push)
↓
```


Audit log entry created

All Events

Event	Publisher	Routing Key
OccupancyUpdated	AI Worker	`events.occupancy_updated`
HelmetMissingDetected	AI Worker	`events.helmet_missing_detected`
VestMissingDetected	AI Worker	`events.vest_missing_detected`
GlovesMissingDetected	AI Worker	`events.gloves_missing_detected`
ShoesMissingDetected	AI Worker	`events.shoes_missing_detected`
OvercrowdingDetected	AI Worker	`events.overcrowding_detected`
UnauthorizedEntryDetected	AI Worker	`events.unauthorized_entry_detected`
CameraOfflineDetected	AI Worker	`events.camera_offline_detected`
CameraReconnected	AI Worker	`events.camera_reconnected`
WorkerHeartbeat	AI Worker	`events.worker_heartbeat`
ModelUpdated	Backend	`config.model_updated`
ConfigUpdated	Backend	`config.zone_config_updated`

Event Payload Example

```
{
  "event": "helmet_missing_detected",
  "event_id": "uuid-v4",
  "camera_id": "CAM-001",
  "zone_id": "ZONE-WELDING",
  "factory_id": "FACTORY-GN",
  "timestamp": "2026-01-01T10:00:00Z",
  "confidence": 0.91,
  "snapshot_key": "snapshots/ZONE-WELDING/CAM-001/2026-01-01T10-00-00.jpg",
  "worker_id": "worker-1"
}
```

Dead Letter Queue (DLQ)

Processing fails
↓
Attempt 1: immediate retry
Attempt 2: 30 seconds

```
Attempt 3: 5 minutes
Attempt 4: → DLQ (manual review)
↓
Background job monitors DLQ depth
DLQ depth > 100 → Prometheus alert → operations team notified
```

8. AI Worker Architecture

Worker Design

Each worker process manages multiple cameras independently:

```
Worker Process
■■■ CameraWorker (CAM-001) – runs in thread/coroutine
■■■ CameraWorker (CAM-002)
■■■ CameraWorker (CAM-003)
■■■ ... up to 20-25 cameras per worker
```

Detection Pipeline (per camera)

```
RTSP Stream
↓
FrameReader – capture + auto-reconnect
↓
Frame Sampling – 2 FPS from 25 FPS (configurable)
↓
YOLO Detector – person + PPE detection
↓
Confidence Threshold Check (zone-configurable)
↓
ByteTrack Tracker – assign track IDs
↓
OccupancyCounter – count persons in zone
↓
PPEValidator – check required PPE per tracked person
↓
RulesEngine – evaluate violation conditions
↓
SnapshotCapture – save frame to MinIO on violation
↓
RabbitMQPublisher – publish event
```

Default Confidence Thresholds

Detection	Default	Configurable
Person	0.70	Per zone
Helmet	0.75	Per zone
Safety Vest	0.75	Per zone
Gloves	0.70	Per zone
Safety Shoes	0.70	Per zone

RTSP Resilience

```
Stream lost
↓
Retry: 5 seconds
Retry: 15 seconds
Retry: 60 seconds
↓ (still offline)
Publish CameraOfflineDetected
Backend creates High severity alert
IT team notified
Worker continues all other cameras
```

Circuit Breaker

```
3 consecutive processing failures on same camera
↓
Camera isolated from worker
CameraOfflineDetected published
Worker continues remaining cameras
```

Worker ↔ Backend Config Sync (Race Condition Safe)

```
Worker starts
↓
Step 1: Pull full config from REST API → /api/v1/config/zone/{zone_id}
(authoritative source, includes current version number)
↓
Step 2: Subscribe to RabbitMQ config.zone_config_updated
↓
ConfigUpdated event received
↓
Compare event.version vs local_version
↓
event.version > local_version → apply + update local_version
event.version <= local_version → discard (stale, already have latest)
```

Model Hot-Swap

```
Admin uploads new model via backend API
↓
New model stored in MinIO model registry
Backend publishes ModelUpdated event
↓
All workers receive event
Worker downloads new model weights
Worker swaps model (stream continues, no restart)
Worker publishes WorkerHeartbeat with new model version
```

Worker Metrics (Prometheus)

```
worker_frames_processed_total – total frames processed
worker_frames_dropped_total – frames dropped (queue overflow)
worker_inference_latency_seconds – per-frame inference time (histogram)
worker_detection_count_total – detections by type
worker_queue_depth – RabbitMQ publish queue depth
worker_camera_reconnect_total – camera reconnection attempts
worker_circuit_breaker_open_total – cameras isolated by circuit breaker
worker_heartbeat_timestamp – last heartbeat (liveness check)
worker_model_version – active model version
```

9. Rules Engine

Rules are **not hardcoded** — they are stored in the database and loaded per zone.

Rule Structure

```
Rule:
id: UUID
zone_id: UUID
condition_type: Enum (HELMET_MISSING, VEST_MISSING, OVERCROWDING, ...)
duration_seconds: int # sustained for N seconds before alert
severity: Enum (CRITICAL, HIGH, MEDIUM, LOW)
enabled: bool
actions: List[Enum] (CREATE_ALERT, SEND_NOTIFICATION, STORE_SNAPSHOT, LOG_AUDIT)
```

Example Rules

```
Rule 1:
IF person_detected AND helmet_missing
FOR 3 seconds
THEN create_alert(severity=HIGH) + store_snapshot + notify_supervisor
```

```

Rule 2:
IF current_count > zone.max_occupancy
FOR 0 seconds (immediate)
THEN create_alert(severity=HIGH) + notify_supervisor

Rule 3:
IF camera_offline
FOR 60 seconds
THEN create_alert(severity=HIGH) + notify_it_team

Rule 4:
IF person_detected IN restricted_zone
FOR 0 seconds
THEN create_alert(severity=HIGH) + store_snapshot + notify_security

```

Rules Engine Evaluation

```

for tracked_person in current_frame_persons:
for rule in zone.active_rules:
if rule.condition_type == HELMET_MISSING:
if not tracked_person.has_helmet:
violation_tracker.record(tracked_person.track_id, rule)
if violation_tracker.duration(tracked_person.track_id) >= rule.duration_seconds:
execute_actions(rule.actions, tracked_person, frame)

```

Duration tracking persists across frames using track IDs from ByteTrack.

10. WebSocket & Real-Time Architecture

Why WebSocket

200 concurrent supervisors polling every 2 seconds = 6,000 requests/minute on backend + DB. WebSocket push eliminates this entirely — the backend pushes only when data changes.

Connection Manager

```

class WebSocketConnectionManager:
connections: Dict[str, List[WebSocket]] # user_id → sockets
role_index: Dict[str, Set[str]] # role → user_ids

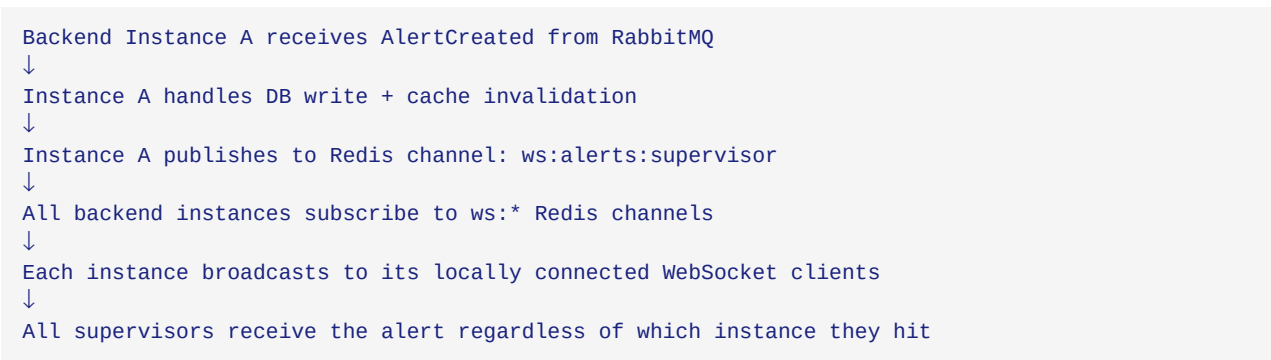
async def connect(user_id, role, websocket)
async def disconnect(user_id, websocket)
async def send_to_user(user_id, message)
async def broadcast_to_role(role, message)
async def broadcast_to_all(message)

```

Channels

Channel	Message Types
`/ws/dashboard`	OccupancyUpdated, CameraStatusChanged, SafetyScoreUpdated
`/ws/alerts`	AlertCreated, AlertAcknowledged, AlertResolved, AlertAssigned

Redis Pub/Sub for Multi-Instance Scaling



Concurrent Connection Limits

Role	Max WebSocket Connections
Viewer	3
Supervisor	5
Safety Officer	5
Admin	10

Exceeded limit → 4008 Too Many Connections close code.

11. API Design & Versioning

URL Structure

```
/api/v1/resource  
/api/v2/resource ← future breaking changes
```

Standard Response Model

```
{  
  "success": true,  
  "data": { ... },  
  "meta": {  
    "page": 1,  
    "page_size": 20,  
    "total": 145,  
    "total_pages": 8  
  },  
  "error": null  
}
```

Error Response

```
{  
  "success": false,  
  "data": null,  
  "error": {  
    "code": "ALERT_NOT_FOUND",  
    "message": "Alert with id abc-123 does not exist",  
    "details": {}  
  }  
}
```

Query Parameters (all list endpoints)

```
?page=1&page_size=20  
?sort_by=created_on&sort_dir=desc  
?filter[status]=Open  
?search=welding
```

Versioning Policy

Change Type	Version Bump
Add optional field to response	No
Add new endpoint	No
Remove or rename field	Yes (v2)
Change field type	Yes (v2)
Remove endpoint	Yes (v2)

Deprecated v1 endpoints return:

```
Deprecation: true
Sunset: 2027-06-01
X-API-Version: v1
```

v1 maintained for minimum 6 months after v2 release.

12. Caching Strategy

Redis is the primary cache layer. Cache is event-driven — invalidated on every write, not on TTL alone.

What is Cached

Key Pattern	Content	TTL
`dashboard:summary:{factory_id}`	Executive dashboard stats	30s
`zone:occupancy:{zone_id}`	Current occupancy count	10s
`camera:status:all`	Camera online/offline list	15s
`alert:count:{zone_id}`	Open alert count per zone	30s
`snapshot:url:{snapshot_key}`	MinIO pre-signed URL	14 min
`user:permissions:{user_id}`	RBAC permission set	1 hour
`zone:config:{zone_id}`	Detection config per zone	until invalidated

Cache Invalidation

```
Backend consumer receives OccupancyUpdated event
↓
occupancy.logs row inserted
↓
Redis.delete("zone:occupancy:{zone_id}")
Redis.delete("dashboard:summary:{factory_id}")
↓
Next dashboard request fetches fresh data and repopulates cache
```

13. Database Architecture

Database: PostgreSQL 16 with native table partitioning.

Schema Overview

```
identity → users, roles, permissions, refresh_tokens
organization → organizations (future multi-tenant)
factory → factories
zone → zones
camera → cameras
worker → ai_workers
occupancy → logs (partitioned by month)
ppe → violations (partitioned by month)
alerts → alerts, alert_history
notifications → notification_log
config → zone_configs
audit → audit_log
analytics → pre-aggregated analytics (materialized views)
```

Core Tables

identity.users

```
id UUID PK
name VARCHAR
email VARCHAR UNIQUE
password_hash VARCHAR
role_id FK → identity.roles
status ENUM (Active, Inactive, Suspended)
created_on TIMESTAMPTZ
last_login_at TIMESTAMPTZ
created_by FK → identity.users
modified_by FK → identity.users
version INT DEFAULT 1
deleted_at TIMESTAMPTZ (soft delete)
```

identity.roles

```
id UUID PK
name VARCHAR UNIQUE (Admin, SafetyOfficer, Supervisor, Viewer)
permissions JSONB
created_on TIMESTAMPTZ
```

identity.refresh_tokens

```
id UUID PK
user_id FK → identity.users
token_hash VARCHAR (SHA-256, never plaintext)
expires_at TIMESTAMPTZ
status ENUM (Active, Revoked)
created_at TIMESTAMPTZ
revoked_at TIMESTAMPTZ
```

factory.factories

```
id UUID PK
name VARCHAR
code VARCHAR UNIQUE
```

```
address TEXT
location VARCHAR
plant_head_id FK → identity.users
status ENUM (Active, Inactive)
created_on TIMESTAMPTZ
created_by FK → identity.users
modified_by FK → identity.users
version INT DEFAULT 1
deleted_at TIMESTAMPTZ
```

zone.zones

```
id UUID PK
factory_id FK → factory.factories
name VARCHAR
code VARCHAR UNIQUE
max_occupancy INT
supervisor_id FK → identity.users
description TEXT
status ENUM (Active, Inactive)
created_on TIMESTAMPTZ
created_by FK → identity.users
modified_by FK → identity.users
version INT DEFAULT 1
deleted_at TIMESTAMPTZ
```

camera.cameras

```
id UUID PK
zone_id FK → zone.zones
name VARCHAR
code VARCHAR UNIQUE
rtsp_url VARCHAR (encrypted at rest)
status ENUM (Online, Offline, Degraded)
fps DECIMAL
last_seen_at TIMESTAMPTZ
worker_id FK → worker.ai_workers
created_on TIMESTAMPTZ
created_by FK → identity.users
modified_by FK → identity.users
version INT DEFAULT 1
deleted_at TIMESTAMPTZ
```

confia.zone confias

```
id UUID PK
zone_id FK → zone.zones UNIQUE
person_threshold DECIMAL DEFAULT 0.70
helmet_threshold DECIMAL DEFAULT 0.75
vest_threshold DECIMAL DEFAULT 0.75
gloves_threshold DECIMAL DEFAULT 0.70
shoes_threshold DECIMAL DEFAULT 0.70
max_occupancy INT
ppe_required JSONB (["helmet", "vest", "gloves"])
updated_by FK → identity.users
updated_at TIMESTAMPTZ
version INT DEFAULT 1 ← incremented on every save
```

Workers use `version` to safely discard stale config events during restarts.

occupancy.loads (partitioned by month)

```
id UUID PK
zone_id FK → zone.zones
camera_id FK → camera.cameras
current_count INT
timestamp TIMESTAMPTZ
```

Partition by: `PARTITION BY RANGE (timestamp)`

Retain 12 months online. Archive older to cold storage.

ppe.violations (partitioned by month)

```
id UUID PK
zone_id FK → zone.zones
camera_id FK → camera.cameras
violation_type ENUM (HELMET_MISSING, VEST_MISSING, GLOVES_MISSING, SHOES_MISSING, OVERCROWDING,
UNAUTHORIZED_ENTRY)
confidence DECIMAL
snapshot_key VARCHAR (MinIO object key, not a local path)
track_id VARCHAR (ByteTrack track ID)
created_on TIMESTAMPTZ
```

alerts.alerts

```
id UUID PK
violation_id FK → ppe.violations (nullable for camera alerts)
factory_id FK → factory.factories
zone_id FK → zone.zones
camera_id FK → camera.cameras
alert_type ENUM
severity ENUM (Critical, High, Medium, Low)
status ENUM (Open, Acknowledged, InProgress, Closed)
assigned_to FK → identity.users
created_on TIMESTAMPTZ
acknowledged_on TIMESTAMPTZ
resolved_on TIMESTAMPTZ
created_by VARCHAR (system or user_id)
```

alerts.alert_history

```
id UUID PK
alert_id FK → alerts.alerts
from_status ENUM
to_status ENUM
changed_by FK → identity.users
changed_at TIMESTAMPTZ
comment TEXT
```

notifications.notification_log

```
id UUID PK
alert_id FK → alerts.alerts
channel ENUM (Email, InApp, WebPush, Slack, Teams, Webhook)
```

```
recipient_id FK → identity.users
sent_at TIMESTAMPTZ
status ENUM (Sent, Failed, Delivered)
failure_reason TEXT
retry_count INT DEFAULT 0
```

audit.audit_log (immutable — no UPDATE/DELETE permitted)

```
id UUID PK
user_id FK → identity.users
action VARCHAR (AlertResolved, ConfigChanged, UserCreated, ...)
entity_type VARCHAR
entity_id UUID
old_value JSONB
new_value JSONB
ip_address VARCHAR
user_agent VARCHAR
correlation_id VARCHAR
timestamp TIMESTAMPTZ
```

Indexes

```
-- Alerts
CREATE INDEX idx_alerts_status ON alerts.alerts(status);
CREATE INDEX idx_alerts_zone_id ON alerts.alerts(zone_id);
CREATE INDEX idx_alerts_created_on ON alerts.alerts(created_on DESC);
CREATE INDEX idx_alerts_severity ON alerts.alerts(severity);

-- Occupancy (partition-aware)
CREATE INDEX idx_occupancy_zone_ts ON occupancy.logs(zone_id, timestamp DESC);

-- Violations (partition-aware)
CREATE INDEX idx_violations_zone ON ppe.violations(zone_id, created_on DESC);
CREATE INDEX idx_violations_type ON ppe.violations(violation_type, created_on DESC);

-- Audit
CREATE INDEX idx_audit_user ON audit.audit_log(user_id, timestamp DESC);
CREATE INDEX idx_audit_entity ON audit.audit_log(entity_type, entity_id);
```

14. Snapshot Storage Architecture

Violation snapshots → MinIO (on-prem) or AWS S3 (cloud).

Key Format

```
snapshots/{factory_id}/{zone_id}/{camera_id}/{YYYY-MM-DD}/{HH-MM-SS}.jpg
```

Pre-Signed URL Strategy

```
Frontend requests alert detail (includes snapshot_key)
↓
Backend checks Redis: snapshot:url:{snapshot_key}
↓ (cache miss)
Backend calls MinIO.presigned_get_url(key, expiry=15min)
↓
URL stored in Redis with TTL = 14 minutes (1 min safety buffer)
↓
URL returned to frontend
↓ (cache hit)
Cached URL returned – no MinIO API call
```

Snapshots are never served from local disk. No direct MinIO URLs exposed to clients.

URL Refresh Endpoint

If a URL expires while the user has the page open:

```
GET /api/v1/snapshots/{snapshot_key}/url
→ Returns a fresh 15-minute pre-signed URL
```

15. Data Retention Policy

Table	Online	Cold Archive	Purge
occupancy.logs	12 months	2 years	After 2 years
ppe.violations	24 months	3 years	After 3 years
alerts.alerts	24 months	3 years	After 3 years
audit.audit_log	36 months	5 years	After 5 years
snapshots (MinIO)	90 days	1 year	After 1 year
identity.refresh_tokens	7 days	—	Auto-purged

Automated: a nightly background job runs partition archival and MinIO lifecycle policy enforces snapshot retention.

16. Dashboard Architecture

Executive Dashboard (Admin / Plant Manager)

Metric	Source
Total Cameras	cache: camera:status:all
Online / Offline Cameras	cache: camera:status:all
Active Alerts	cache: alert:count:*
PPE Compliance %	analytics materialized view
Safety Score	computed from compliance + incident rate
Today's Violations	analytics

Safety Dashboard (Safety Officer)

- Violations by type (today / this week / this month)
- Open alerts with severity breakdown
- Alert resolution time trend
- Violation heatmap by zone

Zone Dashboard (Supervisor)

- Current occupancy vs limit (real-time via WebSocket)
- Current violations in assigned zones
- Camera status for assigned zone
- Recent alerts for zone

Camera Dashboard

- Camera grid with live status
- FPS per camera
- Last seen timestamp
- Worker assignment
- Stream health indicator

17. Analytics Architecture

KPIs

KPI	Calculation
Helmet Compliance %	$(\text{frames with helmet} / \text{total person frames}) \times 100$
Vest Compliance %	$(\text{frames with vest} / \text{total person frames}) \times 100$
PPE Compliance %	average of all PPE compliance metrics
Safety Score	$0.6 \times \text{PPE Compliance} + 0.3 \times (1 - \text{incident rate}) + 0.1 \times \text{alert closure speed}$
Alert Response Time	$\text{avg}(\text{acknowledged_on} - \text{created_on})$
Alert Closure Time	$\text{avg}(\text{resolved_on} - \text{created_on})$
Peak Occupancy	$\text{max}(\text{current_count})$ per zone per day
Overcrowding Frequency	$\text{count}(\text{overcrowding events})$ per zone per week

Trend Periods

- Daily (last 30 days)
- Weekly (last 12 weeks)
- Monthly (last 12 months)

Violation Heatmap

Zone × Hour grid showing violation density. Used to identify high-risk time windows.

Report Export

- PDF: generated server-side using WeasyPrint / ReportLab
- Excel: generated using openpyxl

18. Notification Architecture

Channels

Channel	Library	Use Case
Email	SMTP (SendGrid / self-hosted)	Alert + daily summary
In-App	WebSocket push	Real-time alerts

Channel	Library	Use Case
Web Push	Web Push API	Browser notifications
Slack	Slack Webhook API	Team channel alerts
Microsoft Teams	Teams Webhook API	Enterprise teams
Webhook	HTTP POST	Custom integrations

Notification Templates

Templates are stored in DB — configurable per organization per event type.

```

Template: helmet_missing_alert
Subject: "[HIGH] Helmet Violation – {zone_name} – {timestamp}"
Body: "A helmet violation was detected at {zone_name} by {camera_name}."
View alert: {alert_url}"

```

Delivery Flow

```

Violation detected
↓
Alert created
↓
NotificationService.dispatch(alert)
↓
For each recipient in alert.zone.supervisors:
  For each channel in user.notification_preferences:
    ChannelAdapter.send(template, recipient, alert)
    notification_log.insert(status=Sent/Failed)
↓
Failed delivery:
  Retry 1: +30s
  Retry 2: +2min
  Retry 3: +10min
  All failed → operations team notified

```

19. Zone Configuration Flow

Runtime Config Change (No Restart)

```

Admin updates zone config via PUT /api/v1/config/zone/{zone_id}
↓
config.zone_configs updated (version += 1)
Audit log entry created
↓

```



```
ConfigUpdated event published to RabbitMQ
Event payload includes: zone_id, new_config, new_version
↓
All AI workers subscribed to config.zone_config_updated receive event
↓
Each worker: compare event.version vs local_config.version
↓
event.version > local → apply new config, update local_version
event.version ≤ local → discard (stale)
↓
Detection thresholds + PPE rules updated live
No camera stream restart required
```

20. API Rate Limiting

Limits enforced per user per role using Redis sliding window counter.

Role	Limit
Viewer	60 req/min
Supervisor	120 req/min
Safety Officer	120 req/min
Admin	300 req/min

Rate limit exceeded → 429 Too Many Requests + Retry-After header.

Response headers on every request:

```
X-RateLimit-Limit: 120
X-RateLimit-Remaining: 87
X-RateLimit-Reset: 1735700460
```

WebSocket connections are not subject to HTTP rate limits (governed by connection limits per role).

21. Security Architecture

OWASP Top 10 Coverage

Risk	Mitigation
Injection	SQLAlchemy ORM parameterized queries; no raw SQL
Broken Auth	JWT + refresh rotation + blacklisting; HttpOnly cookies

Risk	Mitigation
XSS	React escapes by default; CSP headers via Nginx
CSRF	SameSite=Strict on cookies; token in header for API calls
Security Misconfiguration	Helmet headers via Nginx; strict CORS allowlist
Sensitive Data Exposure	RTSP URLs encrypted at rest; snapshots behind pre-signed URLs
Broken Access Control	RBAC on every endpoint; permission check in middleware
Logging & Monitoring	Structured JSON logs; Prometheus + Grafana + Loki
SSRF	RTSP URL validation against allowlist before saving
Insecure Deserialization	Pydantic v2 strict parsing on all inputs

Additional

- Input validation: Pydantic v2 on all request bodies
- File upload: JPEG/PNG only, max 5MB, scanned for mime-type mismatch
- Secrets: environment variables only; never committed to repo
- HTTPS enforced in production via Nginx
- All WebSocket connections over WSS

22. Monitoring & Observability

Prometheus — Backend Metrics

```
http_requests_total – by endpoint, method, status_code
http_request_duration_seconds – latency histogram
active_websocket_connections – current live connections
rabbitmq_events_consumed_total – by event type
rabbitmq_events_failed_total – events sent to DLQ
dlq_depth – current DLQ message count
alert_created_total – by severity
alert_resolution_seconds – time open → resolved (histogram)
cache_hit_total – by key prefix
cache_miss_total – by key prefix
```

Prometheus — AI Worker Metrics

```
worker_frames_processed_total
worker_frames_dropped_total
```

```
worker_inference_latency_seconds – p50, p90, p99
worker_detection_count_total – by detection_type
worker_queue_depth
worker_camera_reconnect_total
worker_circuit_breaker_open_total
worker_heartbeat_timestamp
worker_model_version
```

Grafana Dashboards

Dashboard	Key Panels
System Overview	RPS, error rate, p99 latency, active connections
AI Worker Health	Inference latency histogram, frames/sec, queue depth per worker
Alert Operations	Alert volume by hour, resolution time trend, open by severity
Camera Health	Online/offline ratio, reconnection rate, circuit breaker count
DLQ Monitor	DLQ depth over time, failed events by type

Alertmanager Rules

Condition	Severity	Destination
DLQ depth > 100	Critical	PagerDuty
Worker heartbeat missing > 60s	Critical	PagerDuty
API error rate > 5%	High	PagerDuty
Inference latency p99 > 2s	Medium	Slack
Camera offline > 60s	Medium	In-app alert to supervisor

23. Audit Trail

Every sensitive action written to `audit.audit_log`:

Action	Trigger
AlertAcknowledged	Supervisor acknowledges alert
AlertResolved	Supervisor resolves alert

Action	Trigger
AlertReassigned	Alert assigned to different user
ConfigChanged	Zone config updated
UserCreated	New user added
UserDisabled	User account disabled
RoleChanged	User role changed
CameraAdded	Camera registered
CameraRemoved	Camera deleted
ModelUpdated	AI model version changed
LoginFailed	Failed authentication attempt
UnauthorizedAccess	403 response on any endpoint
Logout	Session terminated

Rules:

- Audit log is **append-only** — no UPDATE or DELETE permitted at DB level (row-level security)
- Accessible only to Admin role
- Exported as PDF for compliance audits

24. Logging Strategy

Format

All services emit structured JSON logs:

```
{
  "timestamp": "2026-01-01T10:00:00.123Z",
  "level": "INFO",
  "service": "visionguard-backend",
  "module": "alerts",
  "correlation_id": "req-abc-123",
  "user_id": "user-xyz",
  "message": "Alert resolved",
  "alert_id": "alert-456",
  "duration_ms": 45
}
```

Log Levels

Level	When
ERROR	Unhandled exceptions, DLQ overflow, auth failures
WARNING	High DLQ depth, slow inference, degraded camera
INFO	Normal operations — alert created, config changed
DEBUG	Disabled in production

Correlation IDs

Every HTTP request gets a `X-Correlation-ID` header (generated if not provided).

Propagated to all log entries and downstream service calls within that request scope.

25. Testing Strategy

Backend

Unit Tests → domain entities, business rules, use case handlers
Integration Tests → repository implementations (real PostgreSQL)
API Tests → FastAPI TestClient, full request/response cycle
Coverage target → ≥ 80%

AI Worker

Unit Tests → rules engine logic, frame sampling, config sync
Pipeline Tests → mock camera feed → mock detection → event assertion
Coverage target → ≥ 70%

Frontend

Unit Tests → Zustand store, utility functions
Component Tests → React Testing Library
Coverage target → ≥ 70%

26. CI/CD Pipeline

GitHub Actions — runs on every push to `main` and all pull requests.

```
Pipeline:
lint:
- ruff (Python)
- eslint (TypeScript)

format:
- black (Python)
- prettier (TypeScript)

test:
- pytest (backend + AI worker)
- vitest (frontend)

build:
- docker build backend
- docker build ai-worker
- docker build frontend

push: (main branch only)
- push images to container registry

deploy: (main branch only, manual trigger for prod)
- docker compose pull
- docker compose up --no-deps -d
```

27. Deployment Architecture

Development

```
Single machine (laptop / workstation)

React (Vite dev server)
FastAPI (uvicorn --reload)
PostgreSQL (Docker)
RabbitMQ (Docker)
Redis (Docker)
MinIO (Docker)
AI Worker (CPU mode)
```

Production — 50 Cameras (Initial Target)

```
Server 1 (Application)
■■■ Nginx (reverse proxy + SSL termination)
■■■ React (built static files served by Nginx)
■■■ FastAPI (gunicorn + uvicorn workers)
■■■ PostgreSQL
■■■ RabbitMQ
■■■ Redis
■■■ MinIO
```

■■■ Prometheus + Grafana + Loki

Server 2 (GPU Worker)
■■■ AI Worker 1 (25 cameras)

Server 3 (GPU Worker)
■■■ AI Worker 2 (25 cameras)

Production — 200 Cameras

Server 1 → Nginx + React
Servers 2-3 → FastAPI (load balanced)
Server 4 → PostgreSQL (primary) + Redis + RabbitMQ + MinIO
Servers 5-12 → AI Workers (8 × 25 cameras)
Server 13 → Monitoring (Prometheus + Grafana + Loki)

Redis Pub/Sub used for WebSocket cross-instance broadcast.

28. Scaling Strategy

Cameras	Backend	AI Workers	Notes
50	1 server	2 GPU workers	Initial target
100	1 server	4 GPU workers	
200	2 servers (LB)	8 GPU workers	Redis pub/sub for WS
500	3–5 node cluster	20+ workers	PG read replicas
1000+	API Gateway + cluster	50+ workers	Kubernetes

No code changes required at any scale — horizontal scaling only.

PostgreSQL Scaling Path

100 cameras → single primary
200 cameras → primary + 1 read replica (analytics queries)
500 cameras → primary + 2 read replicas + PgBouncer connection pool
1000 cameras → primary + 3 replicas + PgBouncer + Citus (if needed)

29. Future Integrations

System	Purpose	Phase
SAP	Work order creation on violation	Phase 3

System	Purpose	Phase
HRMS	Worker identity, attendance	Phase 2
Attendance System	Auto-link detected persons to employees	Phase 2
Access Control System	Cross-reference badge entry vs camera entry	Phase 3
Fire Alarm System	Correlate fire alarm with camera feed	Phase 3
SMS Gateway	SMS alerts to supervisors	Phase 2
Email Server	SMTP for email notifications	Phase 1

30. Technology Stack

Category	Technology	License
Frontend	React 18	MIT
	TypeScript	Apache 2.0
	Vite	MIT
	Material UI	MIT
	AG Grid Community	MIT
	Recharts	MIT
	TanStack Query	MIT
	Zustand	MIT
	React Hook Form	MIT
	Zod	MIT
Backend	FastAPI	MIT
	SQLAlchemy 2.0	MIT
	Alembic	MIT
	Pydantic v2	MIT
AI	YOLO (Ultralytics)	AGPL v3
	OpenCV	Apache 2.0
	ByteTrack	MIT
	ONNX Runtime	MIT
	PyTorch	BSD

Category	Technology	License
Database	PostgreSQL 16	PostgreSQL License
Messaging	RabbitMQ	Mozilla PL 2.0
Cache	Redis	BSD 3-Clause
Storage	MinIO	AGPL v3
Monitoring	Prometheus	Apache 2.0
	Grafana	AGPL v3
	Loki	AGPL v3
Infrastructure	Docker	Apache 2.0
	Nginx	BSD 2-Clause
CI/CD	GitHub Actions	Free (public repos)

All open source. Zero licensing cost for self-hosted deployment.

> Note: YOLO (Ultralytics) uses AGPL v3. For commercial closed-source deployments, an Ultralytics Enterprise license is required. Portfolio and internal enterprise use is covered under AGPL v3.

31. Success Criteria

Criterion	Target
Helmet Detection Accuracy	≥ 90%
Occupancy Accuracy	≥ 95%
Alert Latency	≤ 5 seconds
Camera Uptime Monitoring	offline detected within 60s
Camera Support (initial)	50 cameras
Camera Support (future)	200+ cameras
System Availability	99.5%
Real-Time Dashboard	WebSocket-driven, zero polling
Historical Analytics	24 months online
API Test Coverage	≥ 80%
Worker Config Race Condition	safely handled via version comparison
Token Security	blacklisting on logout, refresh rotation
WebSocket Scaling	Redis pub/sub for multi-instance

Criterion	Target
Zero Data Loss	DLQ with 3-attempt retry
Audit Compliance	immutable audit log, all sensitive actions

End of Document

32. Enterprise Hierarchy Architecture

5-Level Hierarchy

```
Platform (VisionGuard AI)
■■■ Enterprise ← {Enterprise Name}, {Enterprise Name 2}, {Enterprise Name 3} (tenant)
■■■ Factory ← AC Factory, WM Factory, Fridge Factory
■■■ Department ← Assembly, Welding, QC, Packaging
■■■ Zone ← Zone A, Zone B, Zone C
■■■ Camera ← CAM-001, CAM-002
```

This hierarchy is the core structural model of the entire platform. Every entity — user, alert, violation, config, report — belongs to a node in this tree.

Why Department Level

BRD v1.0 had only Factory → Zone. Department is added because:

- Large factories have 10–20 zones grouped under departments
- Department Head needs isolated visibility (only their dept)
- Analytics must aggregate at department level (not just zone or factory)
- Alert routing differs per department (Welding dept head ≠ QC dept head)

Database — New Tables

enterprises

```
id UUID PK
name VARCHAR ({Enterprise Name}, {Enterprise Name 2})
code VARCHAR UNIQUE
industry VARCHAR
contact_person VARCHAR
contact_email VARCHAR
```

```
status ENUM (Active, Inactive)
created_on TIMESTAMPTZ
```

departments

```
id UUID PK
factory_id FK → factories
name VARCHAR
code VARCHAR
head_user_id FK → users
status ENUM
created_on TIMESTAMPTZ
```

Updated: factories

```
enterprise_id FK → enterprises ← NEW
```

Updated: zones

```
department_id FK → departments ← NEW
```

Updated: ALL tables (users, alerts, violations, occupancv, audit...)

```
enterprise_id FK → enterprises ← NEW (for tenant isolation)
```

Tenant Isolation Strategy

Every table carries `enterprise_id`. PostgreSQL Row Level Security (RLS) enforces isolation:

```
-- Enable RLS on every table
ALTER TABLE alerts ENABLE ROW LEVEL SECURITY;

-- Policy: users only see their enterprise's data
CREATE POLICY enterprise_isolation ON alerts
USING (enterprise_id = current_setting('app.current_enterprise_id')::uuid);
```

Backend middleware sets this on every DB connection:

```
# Middleware – runs on every authenticated request
async def set_tenant_context(db: Session, user: User):
    db.execute(
        text("SET LOCAL app.current_enterprise_id = :eid"),
        {"eid": str(user.enterprise_id)}
    )
# HO Admin: no filter – sees all factories
# Factory Admin: additional factory_id filter applied in queries
```

Access Control — Scope per Role

Role	Enterprise Scope	Factory Scope	Department Scope	Zone Scope
HO Admin	All factories	All	All	All
Factory Admin	Own enterprise	Own factory only	All in factory	All in factory
Department Head	Own enterprise	Own factory	Own department	All in dept
Safety Officer	Own enterprise	Own factory	All in factory	All in factory
Supervisor	Own enterprise	Own factory	Own department	Assigned zones
Viewer	Own enterprise	Assigned factory	Assigned scope	Assigned scope

Middleware Enforcement

```
class ScopeMiddleware:

    def filter_query(self, query, user: User, model):

        # HO Admin – no additional filter needed (RLS handles enterprise)
        if user.role == Role.HO_ADMIN:
            return query

        # Factory Admin – filter by factory
        if user.role == Role.FACTORY_ADMIN:
            return query.filter(model.factory_id == user.factory_id)

        # Department Head – filter by department
        if user.role == Role.DEPT_HEAD:
            return query.filter(model.department_id == user.department_id)

        # Supervisor – filter by assigned zones
        if user.role == Role.SUPERVISOR:
            return query.filter(model.zone_id.in_(user.assigned_zone_ids))

        # Viewer – filter by assigned scope
        return query.filter(model.factory_id == user.factory_id)
```

Cross-Factory Dashboard (HO Admin)

HO Admin sees an enterprise-wide summary — aggregated across all factories:

```
Enterprise Dashboard API:
GET /api/v1/enterprise/dashboard

Response:
{
  "enterprise": "{Enterprise Name}",
  "factories": [
    {
      "id": "...",
      "name": "AC Factory",
```

```

"safety_score": 94,
"active_alerts": 2,
"cameras_online": 18,
"cameras_offline": 0,
"ppe_compliance": 96.2,
"violations_today": 3
},
{
  "id": "...",
  "name": "WM Factory",
  "safety_score": 87,
  "active_alerts": 7,
  "cameras_online": 22,
  "cameras_offline": 2,
  "ppe_compliance": 89.1,
  "violations_today": 14
}
],
"enterprise_totals": {
  "safety_score": 91,
  "active_alerts": 10,
  "total_cameras": 54,
  "ppe_compliance": 92.8
}
}

```

Factory Admin hitting the same endpoint sees **only their factory** in the response — same API, different data based on role.

Multi-Enterprise Support (Supplying to Other Clients)

Platform supports multiple enterprise clients:

```

Client 1 → enterprise_id: uuid-enterprise-1
■■■ AC Factory (factory_id: uuid-factory-1)
■■■ WM Factory (factory_id: uuid-factory-2)

Client 2 → enterprise_id: uuid-enterprise-2
■■■ {Factory Name 2} (factory_id: uuid-factory-3)
■■■ {City B} (factory_id: uuid-factory-4)

```

Isolation guarantee:

- RLS policy ensures `enterprise_id = current_enterprise_id` on every query
- {Enterprise Name}'s HO Admin cannot see Tata's data even if they share the same platform deployment
- AI workers are assigned per factory — no cross-enterprise camera sharing

Super Admin (Your Internal Team)

A `SUPER_ADMIN` role exists for the VisionGuard platform team:

```

Super Admin
→ Can view all enterprises (for support/debugging)
→ Can create / suspend enterprises
→ Can impersonate any org admin (audit-logged)
→ Never appears in client-facing audit logs
→ Access via separate /superadmin/ URL with extra MFA

```

Super Admin bypasses RLS using a master DB connection (separate pool, separate credentials).

Updated Module List

Two new modules added:

Module: Enterprise

```

api/ → GET /api/v1/enterprise/dashboard
GET /api/v1/enterprise/factories
GET /api/v1/enterprise/analytics
application/ → GetEnterpriseDashboardQuery
GetCrossFactoryAnalyticsQuery
domain/ → Enterprise entity
infrastructure/ → EnterpriseRepository

```

Module: Department

```

api/ → CRUD /api/v1/departments
GET /api/v1/departments/{id}/zones
application/ → CreateDepartmentCommand
GetDepartmentDashboardQuery
domain/ → Department entity
infrastructure/ → DepartmentRepository

```

Updated Folder Structure

```

backend/src/modules/

■■■ enterprise/ ← NEW – cross-factory dashboard, enterprise mgmt
■■■ department/ ← NEW – department CRUD, dept-level analytics
■■■ identity/ ← updated – HO_ADMIN, FACTORY_ADMIN, DEPT_HEAD roles
■■■ factory/ ← updated – enterprise_id FK added
■■■ zone/ ← updated – department_id FK added
■■■ ... (all other modules unchanged)

```

API URL Structure (Updated)

```
# Enterprise level (HO Admin only)
GET /api/v1/enterprise/dashboard
GET /api/v1/enterprise/factories
GET /api/v1/enterprise/analytics
GET /api/v1/enterprise/alerts

# Factory level
GET /api/v1/factories
GET /api/v1/factories/{id}/dashboard
GET /api/v1/factories/{id}/departments

# Department level (NEW)
GET /api/v1/departments
POST /api/v1/departments
GET /api/v1/departments/{id}/zones
GET /api/v1/departments/{id}/dashboard

# Zone, Camera, Alerts – unchanged
# But all filtered by user scope automatically
```

Data Flow — Alert Creation with Hierarchy

```
AI Worker detects helmet violation on CAM-001
↓
Event published: {camera_id, zone_id, factory_id, enterprise_id}
↓
Backend consumer creates alert with full hierarchy context:
enterprise_id = uuid-enterprise-1
factory_id = uuid-factory-1
department_id = uuid-assembly-dept
zone_id = uuid-zone-a
camera_id = uuid-cam-001
↓
Alert routing:
Zone Supervisor → notified (zone scope)
Department Head → notified (dept scope)
Factory Safety Officer → notified (factory scope)
HO Admin → visible on enterprise dashboard
↓
Each user sees the alert only within their scope
HO Admin sees it in cross-factory view
Factory Admin sees it in factory dashboard
Dept Head sees it in department dashboard
Supervisor sees it in zone dashboard
```

End of Enterprise Hierarchy Section

33. Dynamic Branding Architecture

Core Rule

No company name, logo, or brand asset shall be hardcoded anywhere in the platform — UI, emails, PDFs, reports, or filenames. All branding is loaded dynamically from the database at runtime.

Enterprise Branding Fields

```
enterprises table:
name VARCHAR → shown in UI headers, emails, reports
code VARCHAR → used in export filenames
logo_url VARCHAR → MinIO path, served as pre-signed URL
favicon_url VARCHAR → browser tab icon
primary_color VARCHAR → hex color for UI theme (e.g. #E31837)
secondary_color VARCHAR → secondary brand color
tagline VARCHAR → optional subtitle on login page
```

Where Branding is Applied

Location	Field Used
Browser tab title	`{enterprise.name} — VisionGuard AI`
Login page logo	`enterprise.logo_url`
Login page title	`enterprise.name`
Dashboard header logo	`enterprise.logo_url`
Dashboard header title	`enterprise.name`
UI primary color (theme)	`enterprise.primary_color`
Email subject	`{enterprise.name} — Alert Notification`
Email header logo	`enterprise.logo_url`
PDF report header	`enterprise.name` + `enterprise.logo_url`
PDF report footer	`enterprise.name`
Export filenames	`{enterprise.code}_safety_report_2026.pdf`
Audit log entries	`enterprise.name`
Notification templates	`{enterprise_name}` placeholder

Logo Storage

Logos are uploaded by Super Admin or Enterprise Admin and stored in MinIO:


```
logos/{enterprise_id}/logo.png
logos/{enterprise_id}/favicon.ico
```

Served to frontend as pre-signed URLs with 1-hour expiry — same pattern as violation snapshots.

Frontend Implementation

```
// On app load – fetch enterprise context
const { data: enterprise } = useQuery({
  queryKey: ['enterprise'],
  queryFn: () => api.get('/api/v1/enterprise/branding')
})

// Apply dynamically everywhere
<title>{enterprise?.name} – VisionGuard AI</title>
<img src={enterprise?.logo_url} alt={enterprise?.name} />
<h1>{enterprise?.name} Safety Dashboard</h1>
```

Theme Color Application

```
// Apply enterprise brand color as CSS variable
useEffect(() => {
  if (enterprise?.primary_color) {
    document.documentElement.style.setProperty(
      '--brand-primary', enterprise.primary_color
    )
  }
}, [enterprise])
```

Email Template Example

```
Subject: [{enterprise.name}] High Alert – Helmet Missing – {zone.name}

Header: [enterprise.logo_url] {enterprise.name} Safety Platform

Body: A helmet violation was detected at {zone.name}...
```

No company name is hardcoded in any template — `{enterprise.name}` is resolved at send time from DB.

PDF Report Header

```
# Server-side PDF generation
def generate_report_header(enterprise: Enterprise):
    logo = minio.get_presigned_url(enterprise.logo_url)
    return {
        "logo": logo,
        "company_name": enterprise.name, # from DB
    }
```

```
"report_title": f"{enterprise.name} – Monthly Safety Report",
"generated_by": "VisionGuard AI Platform"
}
```

Branding API

```
GET /api/v1/enterprise/branding
→ Returns: name, logo_url, favicon_url, primary_color, secondary_color, tagline

POST /api/v1/enterprise/branding/logo
→ Upload new logo (Super Admin / Enterprise Admin only)

PUT /api/v1/enterprise/branding
→ Update colors, tagline, name
```

End of Dynamic Branding Architecture Section

34. First Time Setup & Onboarding Architecture

34.1 Two-Phase Onboarding

Phase 1 – Super Admin (VisionGuard team)
Creates enterprise + HO Admin user

Phase 2 – HO Admin (Client)
Completes Setup Wizard:
Factory → Department → Zone → Camera

34.2 First Login Detection Flow

```
# Login API response
{
  "access_token": "...",
  "refresh_token": "...",
  "user": {
    "id": "uuid",
    "name": "Admin User",
    "email": "admin@company.com",
    "role": "HO_ADMIN",
    "is_first_login": true, ← force password change
    "setup_completed": false, ← redirect to wizard
    "enterprise": {
      "id": "uuid",
      "name": "{Enterprise Name}",
```

```
"logo_url": "...",
"primary_color": "#1976D2"
}
}
}
```

34.3 Frontend Routing Logic

```
// After login – check flags
const handleLoginSuccess = (response: LoginResponse) => {
  const { user } = response

  // Step 1: Force password change on first login
  if (user.is_first_login) {
    navigate('/change-password', {
      state: { required: true }
    })
    return
  }

  // Step 2: Setup wizard for HO Admin
  if (!user.setup_completed && user.role === 'HO_ADMIN') {
    navigate('/setup-wizard')
    return
  }

  // Step 3: Normal dashboard
  navigate('/dashboard')
}
```

34.4 Setup Wizard — State Management

```
// Zustand store for wizard progress
interface SetupWizardStore {
  currentStep: number // 1-4
  factoryId: string | null // created in step 1
  departmentId: string | null // created in step 2
  zoneId: string | null // created in step 3
  cameraId: string | null // created in step 4

  setStep: (step: number) => void
  setFactoryId: (id: string) => void
  setDepartmentId: (id: string) => void
  setZoneId: (id: string) => void
  setCameraId: (id: string) => void
}
```

34.5 Setup Progress — Resume from Last Step

```
# Backend: GET /api/v1/setup/progress
# Returns last saved step so wizard can resume

class SetupProgressResponse(BaseModel):
    last_completed_step: int
    factory_id: Optional[str]
    department_id: Optional[str]
    zone_id: Optional[str]
    camera_id: Optional[str]
    completed_at: Optional[datetime]
```

34.6 RTSP Test Connection Endpoint

```
# POST /api/v1/cameras/test-connection
# Called from Setup Wizard Step 4

class RTSPTestRequest(BaseModel):
    rtsp_url: str

class RTSPTestResponse(BaseModel):
    connected: bool
    fps: Optional[float]
    resolution: Optional[str]
    error_message: Optional[str]

# Backend uses OpenCV to test:
async def test_rtsp_connection(rtsp_url: str) -> RTSPTestResponse:
    cap = cv2.VideoCapture(rtsp_url)
    if not cap.isOpened():
        return RTSPTestResponse(connected=False,
            error_message="Cannot connect to stream")
    fps = cap.get(cv2.CAP_PROP_FPS)
    w = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
    h = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
    cap.release()
    return RTSPTestResponse(
        connected=True,
        fps=fps,
        resolution=f"{int(w)}x{int(h)}"
    )
```

34.7 Setup Wizard API Endpoints

```
GET /api/v1/setup/progress
→ Returns last completed step + created entity IDs

POST /api/v1/setup/factory
→ Creates factory, saves step 1 progress

POST /api/v1/setup/department
→ Creates department, saves step 2 progress

POST /api/v1/setup/zone
```

→ Creates zone + PPE config, saves step 3 progress

POST /api/v1/setup/camera

→ Creates camera, saves step 4 progress

POST /api/v1/setup/complete

→ Sets setup_completed = true

→ Triggers AI worker auto-assignment

→ Returns dashboard redirect URL

POST /api/v1/cameras/test-connection

→ Tests RTSP URL, returns FPS + resolution

34.8 Auto AI Worker Assignment

```
# On setup completion – auto assign workers to cameras
```

```
async def auto_assign_workers(factory_id: str):  
    cameras = camera_repo.get_by_factory(factory_id)  
    workers = worker_repo.get_available_workers()
```

```
    if not workers:  
        # No workers online – cameras queued  
        # Will be assigned when worker comes online  
        return
```

```
    # Round-robin assignment  
    for i, camera in enumerate(cameras):  
        worker = workers[i % len(workers)]  
        camera.worker_id = worker.id  
        camera_repo.save(camera)
```

```
    # Publish config to assigned workers  
    for worker in workers:  
        rabbitmq.publish(  
            routing_key="config.worker_cameras_updated",  
            body={"worker_id": worker.id,  
                 "cameras": get_worker_cameras(worker.id)}  
        )
```

34.9 Updated DB Fields

```
-- identity.users – add onboarding flags  
ALTER TABLE identity.users ADD COLUMN is_first_login BOOLEAN DEFAULT true;  
ALTER TABLE identity.users ADD COLUMN setup_completed BOOLEAN DEFAULT false;  
ALTER TABLE identity.users ADD COLUMN password_changed_at TIMESTAMPTZ;  
ALTER TABLE identity.users ADD COLUMN invited_by UUID REFERENCES identity.users(id);  
ALTER TABLE identity.users ADD COLUMN invited_at TIMESTAMPTZ;  
  
-- New table: onboarding.setup_progress  
CREATE TABLE onboarding.setup_progress (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
user_id UUID NOT NULL REFERENCES identity.users(id),
enterprise_id UUID NOT NULL REFERENCES enterprises(id),
last_completed_step INT DEFAULT 0,
factory_id UUID REFERENCES factory.factories(id),
department_id UUID REFERENCES department.departments(id),
zone_id UUID REFERENCES zone.zones(id),
camera_id UUID REFERENCES camera.cameras(id),
completed_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

End of Onboarding Architecture Section

35. Security Enhancements Architecture

35.1 Two Factor Authentication (TOTP)

```
# Libraries: pyotp + qrcode

class TOTPService:

    def generate_secret(self, user_id: str) -> str:
        secret = pyotp.random_base32()
        # Store encrypted in identity.totp_secrets
        totp_repo.save(user_id, encrypt(secret))
        return secret

    def generate_qr_code(self, user: User, secret: str) -> str:
        totp = pyotp.TOTP(secret)
        uri = totp.provisioning_uri(
            name=user.email,
            issuer_name="VisionGuard AI"
        )
        return generate_qr_image(uri) # base64 PNG

    def verify_token(self, user_id: str, token: str) -> bool:
        secret = decrypt(totp_repo.get(user_id))
        totp = pyotp.TOTP(secret)
        return totp.verify(token, valid_window=1)

    def generate_backup_codes(self) -> List[str]:
        return [secrets.token_hex(4).upper() for _ in range(8)]
```

35.2 Session Timeout

```
# Redis key: session:active:{user_id}
# TTL = configured timeout per role

class SessionManager:

    def refresh_session(self, user_id: str, role: str):
        ttl = SESSION_TIMEOUTS[role] # role → seconds
        redis.setex(f"session:active:{user_id}", ttl, "1")

    def is_session_valid(self, user_id: str) -> bool:
        return redis.exists(f"session:active:{user_id}")

    def invalidate_session(self, user_id: str):
        redis.delete(f"session:active:{user_id}")
```

Frontend sends keepalive ping every 5 minutes if user is active.

If no keepalive → session expires → WebSocket sends `session_expired` event → frontend redirects to login.

35.3 IP Whitelisting

```
# Middleware – runs before every request
class IPWhitelistMiddleware:

    async def __call__(self, request: Request, call_next):
        user_ip = request.client.host
        enterprise_id = get_enterprise_from_token(request)

        if enterprise_id:
            allowed_ranges = ip_whitelist_repo.get(enterprise_id)
            if allowed_ranges and not is_ip_allowed(user_ip, allowed_ranges):
                audit_log(action="UnauthorizedIPAccess", ip=user_ip)
                raise HTTPException(status_code=403,
                                    detail="Access denied from this IP")
            return await call_next(request)
```

36. Alert Enhancements Architecture

36.1 Escalation Engine

```
# Background job – runs every minute
class EscalationEngine:

    async def run(self):
        open_alerts = alert_repo.get_unacknowledged()

        for alert in open_alerts:
            matrix = escalation_repo.get_matrix(
                enterprise_id=alert.enterprise_id,
                severity=alert.severity
```

```

)
elapsed = (now() - alert.created_on).seconds / 60

for level in matrix.levels:
    if elapsed >= level.trigger_minutes:
        if not already_escalated(alert.id, level.level):
            notify_users(level.notify_roles, alert)
            log_escalation(alert.id, level.level)
            audit_log(action="AlertEscalated", alert_id=alert.id)

```

36.2 False Positive Tracking

```

# When supervisor marks false positive:
class MarkFalsePositiveCommand:
    alert_id: str
    reason: str
    reason_detail: Optional[str]

# Effect:
# 1. Alert status → Closed (FalsePositive)
# 2. violation.is_false_positive = True
# 3. Camera false_positive_count += 1
# 4. Excluded from compliance % calculation
# 5. Audit log entry
# 6. If camera FP rate > 20% → recommendation event published

```

36.3 Alert Snooze Implementation

```

# Redis key: snooze:{camera_id}:{violation_type}
# TTL = snooze duration

class AlertSnoozeService:

    def snooze(self, camera_id: str, violation_type: str,
              duration_minutes: int, reason: str, user_id: str):

        key = f"snooze:{camera_id}:{violation_type}"
        redis.setex(key, duration_minutes * 60, user_id)

        # Save to DB for audit + history
        snooze_repo.save(AlertSnooze(
            camera_id=camera_id,
            violation_type=violation_type,
            snoozed_by=user_id,
            snooze_reason=reason,
            snoozed_until=now() + timedelta(minutes=duration_minutes)
        ))
        audit_log(action="AlertSnoozed", camera_id=camera_id)

    def is_snoozed(self, camera_id: str, violation_type: str) -> bool:
        return redis.exists(f"snooze:{camera_id}:{violation_type}")

```


AI worker checks snooze before publishing event:

```
if not snooze_service.is_snoozed(camera_id, violation_type):
    rabbitmq.publish(event)
```

37. Shift Management Architecture

37.1 Active Shift Detection

```
class ShiftService:

    def get_active_shift(self, factory_id: str) -> Optional[Shift]:
        now_time = datetime.now().time()
        now_day = datetime.now().strftime("%a").upper() # MON, TUE...

        shifts = shift_repo.get_by_factory(factory_id)

        for shift in shifts:
            if now_day in shift.days:
                if shift.start_time <= now_time <= shift.end_time:
                    return shift
        return None
```

37.2 Violation Tagging with Shift

```
# AI worker event → backend consumer
# On violation save:

active_shift = shift_service.get_active_shift(factory_id)

violation = Violation(
    zone_id=zone_id,
    camera_id=camera_id,
    violation_type=violation_type,
    shift_id=active_shift.id if active_shift else None, # ← tag
    ...
)
```

All analytics queries can then GROUP BY shift_id.

38. Camera Maintenance Architecture

38.1 Maintenance Mode

```
# Redis key: maintenance:{camera_id}
# When camera enters maintenance mode:

class MaintenanceModeService:

    def enable(self, camera_id: str, until: datetime,
              reason: str, user_id: str):
        # Set Redis flag
        ttl = (until - now()).seconds
        redis.setex(f"maintenance:{camera_id}", ttl, "1")

        # Update DB
        camera_repo.set_maintenance_mode(camera_id, True, until, reason)
        audit_log(action="CameraMaintenanceEnabled",
                  camera_id=camera_id, user_id=user_id)

    def is_in_maintenance(self, camera_id: str) -> bool:
        return redis.exists(f"maintenance:{camera_id}")
```

AI worker checks before publishing any alert:

```
if not maintenance_service.is_in_maintenance(camera_id):
    rabbitmq.publish(event)
```

39. Notification Enhancements Architecture

39.1 SMS via Twilio

```
from twilio.rest import Client

class SMSNotificationAdapter:

    def send(self, phone: str, message: str) -> bool:
        client = Client(TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN)
        try:
            msg = client.messages.create(
                body=message[:160], # SMS limit
                from_=TWILIO_FROM_NUMBER,
                to=phone
            )
            return msg.status in ("queued", "sent")
        except Exception as e:
            log.error(f"SMS failed: {e}")
            return False
```

39.2 Notification Digest

```
# Background job – runs every 15 min / 1 hour based on user preference
class DigestJob:
```

```
async def run(self):
    users_with_digest = user_repo.get_digest_preference_users()

    for user in users_with_digest:
        pending = notification_queue_repo.get_pending(user.id)
        if not pending:
            continue

        digest = build_digest(pending)
        email_adapter.send(user.email, digest)
        notification_queue_repo.mark_sent(user.id)
```

39.3 DND Implementation

```
class DNDService:

    def is_dnd_active(self, user: User) -> bool:
        if not user.dnd_enabled:
            return False
        now_time = datetime.now().time()
        return user.dnd_start <= now_time or now_time <= user.dnd_end

    def should_notify(self, user: User,
                      severity: AlertSeverity) -> bool:
        # Critical always bypasses DND
        if severity == AlertSeverity.CRITICAL:
            return True
        return not self.is_dnd_active(user)
```

40. Config Enhancements Architecture

40.1 Config History Tracking

```
# Every zone config save:
class SaveZoneConfigCommand:

    async def handle(self, zone_id: str,
                     new_config: ZoneConfig, user_id: str):

        old_config = config_repo.get(zone_id)

        # Save new config
        new_config.version = old_config.version + 1
        config_repo.save(new_config)

        # Save history
        config_history_repo.save(ConfigHistory(
```

```
zone_id=zone_id,
changed_by=user_id,
old_config=old_config.to_dict(),
new_config=new_config.to_dict(),
changed_at=now()
))

# Publish to AI workers
rabbitmq.publish("config.zone_config_updated", {
    "zone_id": zone_id,
    "version": new_config.version,
    "config": new_config.to_dict()
})
```

40.2 Config Templates

```
# Apply template to zone
class ApplyConfigTemplateCommand:

    async def handle(self, template_id: str,
zone_ids: List[str], user_id: str):

        template = template_repo.get(template_id)

        for zone_id in zone_ids:
            # Apply template config to each zone
            await save_zone_config(zone_id,
template.config_data, user_id)
```

41. New Modules Added

Module: shifts

```
api/ → CRUD /api/v1/shifts
GET /api/v1/shifts/active (current shift)
application/ → CreateShiftCommand, GetActiveShiftQuery
domain/ → Shift entity
infrastructure/ → ShiftRepository
```

Module: maintenance

```
api/ → CRUD /api/v1/maintenance
POST /api/v1/cameras/{id}/maintenance/enable
POST /api/v1/cameras/{id}/maintenance/complete
application/ → EnableMaintenanceModeCommand
CompleteMaintenanceCommand
domain/ → CameraMaintenance entity
```

```
infrastructure/ → MaintenanceRepository
```

Module: announcements

```
api/ → CRUD /api/v1/announcements
POST /api/v1/announcements/{id}/read
application/ → CreateAnnouncementCommand
MarkAnnouncementReadCommand
domain/ → Announcement entity
infrastructure/ → AnnouncementRepository
```

Module: api_keys

```
api/ → POST /api/v1/api-keys
GET /api/v1/api-keys
DELETE /api/v1/api-keys/{id}
application/ → GenerateApiKeyCommand, RevokeApiKeyCommand
domain/ → ApiKey entity
infrastructure/ → ApiKeyRepository
```

42. Updated Complete Module List

```
backend/src/modules/
```

```
■■■ identity/ Auth, JWT, 2FA, sessions, users, roles
■■■ organization/ Multi-tenant support
■■■ enterprise/ Cross-factory dashboard, branding
■■■ factory/ Factory management
■■■ department/ Department management
■■■ zone/ Zone management
■■■ camera/ Camera registry, health, maintenance mode
■■■ worker/ AI worker registry, heartbeat, model mgmt
■■■ occupancy/ Real-time occupancy tracking
■■■ ppe/ PPE violation records
■■■ alerts/ Alert lifecycle, escalation, snooze, bulk actions
■■■ incidents/ Incident management
■■■ notifications/ Email, SMS, in-app, push, Slack, Teams, webhook
■■■ analytics/ KPIs, trends, heatmaps, shift analytics
■■■ dashboard/ Dashboard aggregation, activity feed
■■■ reports/ PDF, Excel, scheduled, comparative
■■■ config/ Zone config, templates, history, bulk update
■■■ audit/ Immutable audit log
■■■ health/ Readiness, liveness, system status
■■■ shifts/ Shift management ← NEW
■■■ maintenance/ Camera maintenance ← NEW
■■■ announcements/ Notice board ← NEW
■■■ api_keys/ Client API access ← NEW
```

End of Enhancement Architecture Sections